

MARK HANEY

STAFF SOFTWARE ENGINEER | GO BACKEND ENGINEER | CLOUD MICROSERVICES | KUBERNETES | AWS

Dardanelle, AR

<https://www.linkedin.com/in/markhaney-33b675ba>

+1 (430) 279-2231 | markhaney.dev@outlook.com

PROFESSIONAL SUMMARY

Staff Software Engineer specializing in **Go 1.20–1.22**, cloud-native backend platforms, distributed systems, and high-throughput APIs across SaaS, property technology, nonprofit software, civic technology, and managed IT industries. Strong experience modernizing legacy services, building **gRPC/REST microservices**, improving CI/CD reliability, tuning PostgreSQL and Redis workloads, and running production systems on **Kubernetes**, AWS, and GCP. Known for solving race conditions, memory leaks, deployment failures, data consistency issues, and observability gaps while mentoring engineers and improving performance, uptime, release quality, and long-term maintainability.

TECH SKILLS

- ❖ **Languages:** **Go 1.20–1.22**, Go 1.14–1.19 migration, TypeScript, JavaScript, SQL, Bash
- ❖ **Backend Engineering:** **gRPC**, REST APIs, microservices, event-driven architecture, API gateway design, service discovery, background workers
- ❖ **Cloud & DevOps:** AWS, GCP, Docker, **Kubernetes**, Terraform, GitHub Actions, GitLab CI, Jenkins, Helm
- ❖ **Databases & Messaging:** **PostgreSQL**, MySQL, MongoDB, Redis, Kafka, RabbitMQ, Elasticsearch
- ❖ **Testing & Quality:** Go testing package, Testify, GoMock, integration testing, contract testing, load testing, code review
- ❖ **Observability:** **OpenTelemetry**, Prometheus, Grafana, CloudWatch, ELK, structured logging, distributed tracing
- ❖ **Architecture:** Distributed systems, domain-driven design, clean architecture, system design, scalability, reliability engineering

PROFESSIONAL EXPERIENCE

Staff Software Engineer

Relatio

Jun 2022 – Apr 2026

Las Vegas, NV

- ❖ Led backend architecture for a SaaS data intelligence platform using **Go 1.20–1.22**, **gRPC**, REST, PostgreSQL, Redis, Kafka, and Kubernetes to support reliable multi-tenant product workflows.
- ❖ Designed event-driven microservices that separated ingestion, validation, enrichment, and notification logic, making the platform easier to scale and safer to release.
- ❖ Migrated several legacy background jobs into **Go-based workers**, replacing fragile cron behavior with idempotent consumers, retry policies, and dead-letter handling.
- ❖ Resolved a race condition in concurrent workflow execution by adding optimistic locking and idempotency keys, reducing duplicate processing incidents by **27%**.
- ❖ Improved slow PostgreSQL queries by redesigning indexes, batching high-volume reads, and adding Redis caching, cutting average API latency.
- ❖ Reworked CI/CD pipelines with GitHub Actions, Docker image scanning, and staged Kubernetes rollouts, reducing failed production deployments by **24%**.

- ❖ Implemented **OpenTelemetry** tracing across Go services to connect API requests, Kafka messages, and database calls into one debuggable request path.
- ❖ Built feature flag support for high-risk backend changes, allowing product teams to release gradually while improving deployment confidence.
- ❖ Fixed a memory leak caused by unbounded goroutine creation in streaming jobs by adding worker pools, context cancellation, and backpressure controls.
- ❖ Strengthened authentication and authorization around tenant-scoped APIs by improving JWT validation, permission checks, and audit logging for sensitive actions.
- ❖ Mentored backend engineers on **Go concurrency**, clean architecture, testing strategy, and production debugging during design reviews and incident reviews.
- ❖ Partnered with product, security, and platform teams to balance delivery speed with maintainability, compliance needs, and long-term service ownership.

Senior Software Engineer

AppFolio

Feb 2018 – May 2022

Goleta, CA

- ❖ Built property technology backend services using **Go 1.14–1.18**, PostgreSQL, Redis, Docker, AWS, and Kubernetes to support leasing, payments, and workflow automation.
- ❖ Modernized selected monolith workflows into independent **Go microservices**, improving ownership boundaries while keeping customer-facing behavior stable during migration.
- ❖ Implemented payment status synchronization with idempotent REST endpoints, background retries, and reconciliation jobs to prevent missing or duplicated transaction states.
- ❖ Reduced tenant search response time by **29%** through query plan tuning, better pagination, selective indexes, and Redis-backed hot-path caching.
- ❖ Improved batch document processing throughput by **34%** by replacing sequential workers with bounded goroutine pools and channel-based coordination.
- ❖ Solved production timeout issues in high-traffic APIs by adding request context deadlines, connection pool tuning, and safer dependency timeout handling.
- ❖ Migrated older dependency management into **Go Modules**, standardizing builds across local development, CI pipelines, and containerized deployment environments.
- ❖ Expanded automated testing with table-driven unit tests, integration tests, and mock-based service boundaries, increasing backend test coverage.
- ❖ Added Prometheus metrics and structured logs around payment and lease workflows, making incidents easier to isolate during production support rotations.
- ❖ Collaborated with frontend and QA teams to deliver customer-facing features while protecting API compatibility, data accuracy, and release stability.

Software Engineer

Blackbaud

Jul 2016 – Jan 2018

Charleston, SC

- ❖ Developed nonprofit software backend services using **Go 1.7–1.9**, REST APIs, PostgreSQL, MongoDB, RabbitMQ, Jenkins, and AWS-hosted Linux environments.
- ❖ Created donor data import APIs with validation, batching, and failure reporting, helping customers process large files without blocking daily platform operations.

- ❖ Migrated selected scripting-heavy backend processes into compiled **Go services**, improving runtime predictability and simplifying production deployment steps.
- ❖ Reduced recurring import failures by **21%** by adding schema validation, row-level error reporting, retry logic, and safer transaction boundaries.
- ❖ Improved API performance by **18%** after profiling slow handlers, removing unnecessary database calls, and caching stable lookup data in memory.
- ❖ Fixed message queue ordering issues by separating command processing from notification publishing and adding consistent retry behavior for failed events.
- ❖ Built Jenkins deployment steps for Go binaries with environment-specific configuration, health checks, and rollback notes for safer release execution.
- ❖ Collaborated with support engineers to investigate customer-reported data mismatches, trace root causes, and ship permanent fixes instead of temporary patches.
- ❖ Participated in architecture discussions for service boundaries, database design, and maintainable API contracts across fundraising and engagement workflows.

Software Developer

Granicus

May 2014 – Jun 2016

Dallas, TX

- ❖ Built civic technology backend components using **Go**, REST services, PostgreSQL, MySQL, RabbitMQ, Linux, and Jenkins for government communication workflows.
- ❖ Implemented notification processing services that handled subscription updates, email delivery states, and retryable queue failures for public-sector communication systems.
- ❖ Reworked slow reporting queries with better joins, filtered indexes, and asynchronous export jobs, improving report generation speed.
- ❖ Reduced background job failure rates by adding structured logging, retry limits, queue visibility, and operational runbook notes.
- ❖ Solved a production issue where duplicate notifications were sent by introducing deduplication keys and safer message acknowledgment behavior.
- ❖ Supported migration from manually deployed Linux services to Jenkins-driven build and release jobs, improving repeatability across staging and production.
- ❖ Added API-level validation and clear error responses for invalid citizen subscription data, reducing support escalations related to bad inputs.
- ❖ Worked closely with senior engineers to review code, debug incidents, and strengthen backend reliability for high-volume government communication features.

Junior Software Developer

Vology

Jun 2013 – Apr 2014

Gainesville, FL

- ❖ Supported managed IT platform development using **Go** utilities, Bash, Linux, MySQL, REST integrations, and Jenkins-assisted deployment workflows.
- ❖ Built internal automation tools that collected device, ticket, and service status data, reducing manual operational reporting effort.
- ❖ Fixed recurring data sync errors by improving API retry behavior, logging failed payloads, and adding validation before writing records into MySQL.

- ❖ Helped convert small operational scripts into maintainable Go command-line tools, improving execution speed and reducing script failure noise.
- ❖ Assisted senior developers with bug investigation, code reviews, release preparation, and documentation for internal service management applications.

EDUCATION

Arkansas State University

Bachelor's degree, Computer Science (Sep 2009 – May 2013)